

Types of Operating Systems

1. Overview

An operating system (OS) is software that manages computer hardware and software resources. It acts as a bridge between users and the computer, ensuring smooth operation. Different types of OS serve different needs; some handle one task at a time, while others manage multiple users or real-time processes.

2. Batch Operating System

A Batch Operating System is designed to handle large groups of similar jobs efficiently. It does not interact with the computer directly but instead processes jobs that are grouped by an operator. These jobs are queued and executed one after the other, without user interaction during the process.

Key point: Executes a group of similar jobs automatically in batches without user interaction.

Advantages

- **Minimal Idle Time:** The system minimizes idle time by processing jobs in a continuous sequence without human intervention.
- **Handling Repetitive Tasks:** Ideal for managing large, repetitive tasks, such as payroll and billing, with minimal effort.
- **Improved Throughput:** Batch systems can handle high volumes of jobs at once, improving overall system throughput.

Disadvantages

- **Inefficient CPU Utilization:** When a job is waiting for input/output (I/O), the CPU remains idle, leading to poor utilization of resources.
- **Increased Response Time:** The time between job submission and output can be high as all jobs are processed sequentially.
- **Lack of Real-Time Feedback:** Users cannot interact with the system in real-time, making it less suitable for interactive tasks.

Examples: Insurance Claim Processing; Library Book Records; Stock Market Reports.

3. Multi-Programming Operating System

In a Multi-Programming Operating System, multiple programs run in memory at the same time. The CPU switches between programs, utilizing its resources more effectively and improving overall system performance.

Key point: Runs multiple programs in memory at the same time to maximize CPU usage.

Advantages

- **Better CPU Utilization:** CPU stays busy by switching to another job during I/O wait.
- **Improved Throughput:** Multiple jobs run concurrently, increasing work done per unit time.
- **Efficient Resource Use:** CPU, memory, and I/O devices are shared effectively among processes.

Disadvantages

- **Complex Design:** Requires advanced memory management and CPU scheduling.
- **Security Issues:** More programs in memory increase chances of unauthorized access.
- **High Memory Requirement:** Needs larger RAM to run multiple programs together.

Examples: Banking systems; Railway servers; Billing machines.

4. Multi-tasking / Time-sharing Operating Systems

Multitasking OS is a type of multiprogramming system where each process runs in a round-robin manner. Every task gets a fixed time slice called a quantum. After the quantum ends, the OS switches to the next task, allowing multiple tasks—whether from one user or many—to run smoothly on a single system.

Key point: Allows multiple tasks to run by giving each a small time slice for smooth, shared system use.

Advantages

- Equal CPU Access: Each task gets a fair share of CPU time.
- Reduced Software Duplication: Many users can run the same software without needing separate copies.
- Low CPU Idle Time: Efficient scheduling keeps the CPU busy.

Disadvantages

- Lower Reliability: System failures affect all users.
- Security Concerns: Multiple users increase risks to data integrity and privacy.
- Communication Issues: Data sharing between users can cause conflicts.

Examples: IBM VM/CMS; TSO (Time Sharing Option); Windows Terminal Services.

5. Multi-Processing Operating System

Multi-Processing OS is a type of Operating System in which more than one CPU is used for the execution of resources. It better the throughput of the System.

Key point: Multiprocessing operating system uses two or more CPUs simultaneously to increase speed and reliability.

Advantages

- Faster Processing: Multiple CPUs work simultaneously, increasing overall system speed.
- High Reliability: If one processor fails, others can continue working (fault tolerance).
- Supports Heavy Tasks: Ideal for computation-intensive applications like scientific or industrial tasks.

Disadvantages

- High Cost: Multiple processors and complex hardware increase system cost.
- Complex Design: Requires advanced OS support for communication and task distribution.
- Not Always Efficient: Poor task distribution can lead to idle processors and wasted resources.

Examples: UNIX; Linux (Ubuntu, Red Hat, Debian); macOS.

6. Distributed Operating System

Distributed operating systems connects multiple independent computers through a shared communication network. Each system has its own CPU and memory but works together as a single unit. The main benefit is remote access, allowing users to use files and software stored on other connected systems.

Key point: Connects multiple independent computers to function as a single coordinated system.

Advantages

- Independent Systems: Failure of one machine does not affect others.
- Easily Scalable: New systems can be added to the network easily.
- Lower Processing Delays: Tasks are handled faster across multiple machines.

Disadvantages

- Network Dependency: If the main network fails, communication stops.
 - Lack of Standardization: No well-defined language or model for building such systems.
 - High Cost & Complexity: Hardware is expensive, and the software is complex and not widely understood.
-

Issues With Distributed Operating System

- Networking causes delays in the transfer of data between nodes of a distributed system. Such delays may lead to an inconsistent view of data located in different nodes and make it difficult to know the chronological order in which events occurred in the system.
- Control functions like scheduling, resource allocation and deadlock detection have to be performed in several nodes to achieve computation speedup and provide reliable operation when computers or networking components fail.
- Messages exchanged by processes present in different nodes may travel over public networks and pass through computer systems that are not controlled by the distributed operating system. An intruder may exploit this feature to tamper with messages or create fake messages to fool the authentication procedure and masquerade as a user of the system.

Examples: LOCUS; MICROS; Amoeba.

7. Network Operating System

A Network Operating System (NOS) runs on a server and manages data, users, security, applications, and other network functions. It allows shared access to files, printers, and resources within a small private network. Users can see the configuration and connections of other users, which is why these systems are considered tightly coupled systems.

Key point: Manages and supports users, data, and security across connected computers within a network.

Advantages

- Centralized and Stable Servers: Provide reliable management of resources.
- Easy Upgrades: New hardware and technologies can be added without difficulty.
- Remote Access: Users can access the server from different locations and devices.

Disadvantages

- High Server Cost: Setting up and maintaining servers is expensive.
- Dependency on Server: Most operations rely on a central server.
- Regular Maintenance Needed: Frequent updates and technical support are required.

Examples: Microsoft Windows Server 2003; UNIX, Linux; Mac OS X.

8. Real-Time Operating System

These types of OSs serve real-time systems. The time interval required to process and respond to inputs is very small. This time interval is called response time. Real-time systems are used when there are time requirements that are very strict like missile systems, air traffic control systems, robots, etc.

Types of Real-Time Operating Systems

Type	Description
Hard Real-Time Operating System	Used where strict timing is essential and any delay is unacceptable, such as airbags or automatic parachutes. These systems avoid virtual memory to ensure immediate response.
Soft Real-Time Operating System	Used where timing is important but minor delays are acceptable. These systems aim to give quick and predictable responses but do not require perfect accuracy. They are commonly used in multimedia applications, gaming, video streaming, and other interactive tasks.

Key point: Processes data and responds immediately within strict time limits for real-time applications.

Advantages

- Maximum Consumption: Maximum utilization of devices and systems, thus more output from all the resources.
- Error-Free: These types of systems are error-free.
- Memory Allocation: Memory allocation is best managed in these types of systems.

Disadvantages

- **Limited Tasks:** Very few tasks run at the same time and their concentration is very less on a few applications to avoid errors.
- **Complex Algorithms:** The algorithms are very complex and difficult for the designer to write.
- **Thread Priority:** It is not good to set thread priority, as these systems are much less prone to switching tasks.

Examples: Scientific experiments; Medical imaging systems; Robots.

9. Mobile Operating Systems

Mobile operating systems are designed specifically for mobile devices such as smartphones and tablets. Examples of such operating systems are Android and iOS. These operating systems manage the hardware and software resources of the device, providing a platform for running applications and ensuring a seamless user experience.

Key point: Runs on smartphones and tablets, managing hardware, apps, and touch-based interfaces.

Advantages

- **User-Friendly Interfaces:** Mobile operating systems are designed to be intuitive and easy to use, making them accessible to a wide range of users.
- **Extensive App Ecosystems:** The availability of a vast number of applications allows users to customize their devices to meet their specific needs.
- **Connectivity Options:** Mobile operating systems support multiple connectivity options, enabling users to stay connected wherever they go.

Disadvantages

- **Battery Life Constraints:** Despite advancements in power management, battery life remains a challenge for mobile devices, especially with heavy usage.
- **Security Risks:** Mobile devices are susceptible to various security threats, such as malware and phishing attacks, which can compromise user data.
- **Fragmentation:** In the case of Android, the wide range of devices and customizations can lead to fragmentation, making it difficult for developers to ensure compatibility across all devices.

Examples: Android; iOS; Blackberry.
